PRISM: An Integrated OSINT Platform with Graph-Based Intelligence Correlation and Isolated Sandbox Analysis

Yash Patil, Soumitra Bapat, Sharvari Jadhav
Department of Information Technology
Walchand College of Engineering, Sangli
Shivaji University, Kolhapur, India
yashkiran2004@gmail.com

Mrs. B. S. Shetty
Project Guide, Professor
Department of Information Technology
Walchand College of Engineering, Sangli
Shivaji University, Kolhapur, India

***Abstract*—PRISM is a tool that helps us make sense of the data we collect from the internet. Nevertheless, We use tools like SpiderFoot to gather data. Furthermore, It is hard to understand what it means. We face two problems when we use these tools. Additionally, The first problem is that the tools do not show us all the connections between things. For example if thing A is connected to thing B and thing B is connected to thing C the tool will not show us that thing A is connected to thing C. The second problem is that when we find a website we cannot safely look at it without risking our computer. Hence, PRISM solves these problems. It uses a way of showing connections between things called a graph-based correlation engine. Nevertheless, This engine uses methods like Louvain community detection, z-score anomaly detection and PageRank ranking to find connections. PRISM also has a sandbox that lets us look at websites safely. Moreover, The sandbox uses a container that deletes itself after we are done with it so our computer is not at risk. We tested PRISM on three domains and five PhishTank targets. The results were very good. Consequently, We also tested 100 known websites in the sandbox. Moreover, None of them were able to reach our computer. The code for PRISM is available for anyone to use. Nonetheless,**

***Index Terms*—OSINT, Graph Theory, Community Detection, Sandboxing, Threat Intelligence, Cybersecurity, Risk Scoring, NetworkX**

I. Introduction

A. Background

Open Source Intelligence (OSINT) is about gathering data from the internet. We use OSINT for investigations like tracking phishing campaigns. We also use it to map the infrastructure of actors and to analyze malware. Consequently, Collecting data is not enough. We need to be able to understand what it means and find connections between things. When we use a computer we can accidentally download things. This can lead to problems like stolen login details. Additionally, PRISM helps us avoid these problems by letting us look at websites safely.

B. Problem Statement

We face two problems when we use OSINT tools. Nonetheless, The first problem is that the tools do not show us all the connections between things. The second problem is that when we find a website we cannot safely look at it without risking our computer.

C. Contributions

PRISM has three features. The first feature is a graph-based correlation engine. Furthermore, This engine turns the data we collect into a graph. Moreover, Then uses special methods to find connections. The second feature is a sandbox that lets us look at websites safely. Hence, The third feature is a risk scoring system that gives each thing a score based on how risky it is.

D. Paper Organization

The rest of this paper is organized into sections. Furthermore, Section II talks about related work. Section III explains the algorithms we use. Section IV explains the system. Section V shows the results of our tests. Section VI talks about the results. Section VII suggests future work.

II. Related Work

A. Existing OSINT Tools

There are tools that do similar things to PRISM. Maltego Furthermore, [1] was one of the first tools to use graph visualization. Recon-ng Moreover, [2] is a scriptable tool. SpiderFoot [3] is a tool that has limited rule-based correlations. Therefore, TheHive [4] is a tool that handles incident case management.

B. Graph Methods in Security Research

Louvain community detection [5] has been used to group malware families [6] and identify botnets [7]. PageRank Hence, [8] and betweenness centrality [9] are widely used in attack graph and network topology research. Z-score analysis [11] is a technique for finding anomalies. Consequently,

C. Sandboxing Approaches

Cuckoo Furthermore, [13] is a tool that analyzes malware. Therefore, Joe Sandbox Additionally, [14] and Any.run [15] offer cloud-based analysis. These tools have some problems. They can be hard to set up. They may not be suitable for all investigations.

III. Methodology

A. Graph-Based Correlation Architecture

We use a graph-based correlation engine to find connections between things. Nonetheless, The engine turns the data we collect into a graph. Nevertheless, Then uses special methods to find connections. We use a directed multigraph to represent the data. Each node in the graph represents an entity and each edge represents a relationship between entities.

1) Graph Construction: We turn the data we collect into a graph. We use a lookup table to classify each edge in the graph. Fig. 1 shows the full process from raw scan data to the knowledge graph.

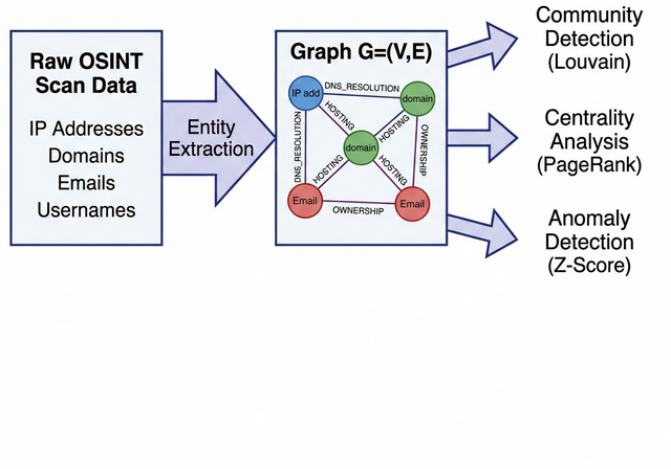


Fig. 1: Graph Construction Pipeline: Raw data is turned into a multi-directed graph $G = (V, E)$ and then passed into three separate analysis tools.

2) Community Detection: We use the Louvain algorithm [5] to find communities in the graph. The algorithm maximizes modularity by alternating between two phases.

$$\Delta Q = \left[\frac{\Sigma_{in} + k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m}\right)^2\right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m}\right)^2 - \left(\frac{k_i}{2m}\right)^2\right] \tag{1}$$

3) Anomaly Detection: We identify four types of issues in the entity graph. We flag isolated nodes, articulation points, high-degree outliers and infrastructure overlap.

4) Centrality Measures: We use PageRank to determine the importance of each entity:

$$PR(i) = \frac{1-d}{N} + d \sum_{j \in B_i} \frac{PR(j)}{L(j)} \tag{2}$$

We also use betweenness centrality:

$$C_B(i) = \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}} \tag{3}$$

And closeness centrality:

$$C_C(i) = \frac{n-1}{\sum_j d(i,j)} \tag{4}$$

B. Sandbox Architecture

Consequently, We use a sandbox to let us look at websites safely. Consequently, The sandbox uses a container that deletes itself after we are done with it. We assume that any website we process could be dangerous. Nevertheless, The analyst needs to be able to see a display and check the browser console without risking the host system.

1) Threat Model and Goals: We assume that any website we process could be dangerous.

2) Design Decisions: We make five design decisions to make the sandbox reliable. We use isolation, short lifetime, no persistent storage, resource caps and optional network isolation. Nonetheless,

1) **Isolation**: Each URL gets its own new container.
2) **Short lifetime**: The container runs for no more than 60 seconds.
3) **No persistent storage**: The container only writes to a shared output directory.
4) **Resource caps**: We set limits on CPU and memory.
5) **Optional network isolation**: The container can be run with restricted network access or through a proxy.

3) Analysis Lifecycle: The analyst enters a URL. Consequently, PRISM's backend creates a new Docker container. The container runs with `--rm` Furthermore, so it automatically deletes itself when the script ends. Additionally, Fig. 2 shows the sandbox architecture.

C. Risk Scoring

Every entity in a completed scan gets a risk score R, calculated from five factors:

$$R = w_1 \cdot T + w_2 \cdot G + w_3 \cdot V + w_4 \cdot I + w_5 \cdot E \tag{5}$$

The five factors are:

- T (Threat Intelligence) — hits from known indicator lists and reputation feeds.
- G (Graph Importance) — centrality scores from the analysis of the graph structure discussed earlier. Nevertheless,
- V (Vulnerability Exposure) — data from scanning modules related to Common Vulnerabilities and Exposures (CVEs).

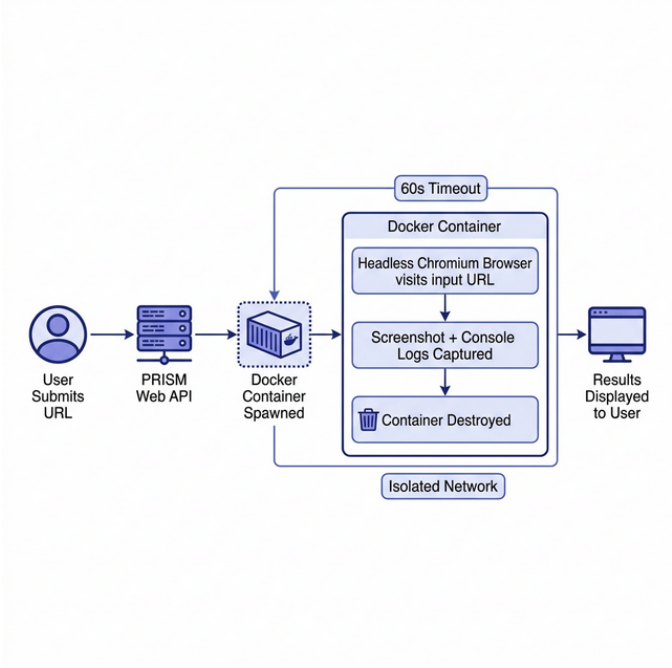


Fig. 2: Isolated Sandbox Architecture: Docker-based URL analysis using containers with a 60-second hard timeout and optional network isolation.

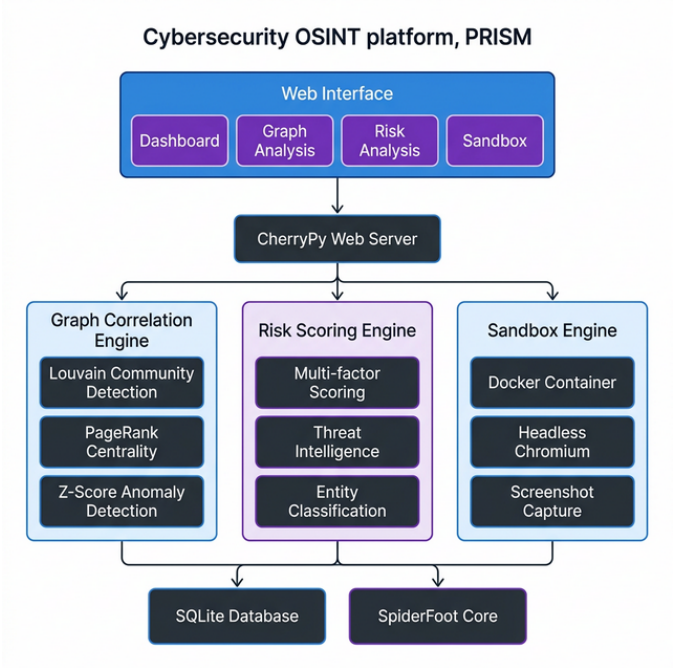


Fig. 3: PRISM System Architecture showing the Web Interface, CherryPy Web Server, three novel engines (Graph Correlation, Risk Scoring, Sandbox) and the SQLite/SpiderFoot core.

- I (Infrastructure Reputation) — trustworthiness of the hosting environment, Autonomous System Number (ASN) flags and provider reputation.
- E (Data Exposure) — leaked credentials and exposed service banners. Consequently,

Intelligence is very important when it comes to phishing cases while the importance of graphs is key in mapping out infrastructure. Therefore,

The `EntityRiskScorer` gives a score of zero to any dimension that does not have supporting data. Furthermore, This prevents scores from being inflated for entities that have limited information.

Final scores are divided into four categories: LOW which is from 0 to 25, MEDIUM which is from 25 to 50, HIGH which is from 50 to 75 and CRITICAL which is from 75 to 100.

IV. Implementation

A. System Architecture

We built PRISM directly on top of SpiderFoot Furthermore, [3] instead of creating a separate version. New modules work alongside existing ones, use the database and connect to the web interface. This means all 200+ existing SpiderFoot modules work without any changes. Fig. 3 shows where the new components fit. Additionally,

Here is a summary of the stack:

- **Backend**: Python 3.7 or higher, CherryPy
- **Database**: SQLite with write-ahead logging mode
- **Graph library**: NetworkX 2.6.3
- **Containerization**: Docker 20.10 or higher
- **Browser automation**: Selenium WebDriver, Chromium
- **Frontend**: JavaScript, Bootstrap 3, D3.js version 5 for graph visualization

B. Module Overview

Table I lists the files that we added and the existing files that we modified heavily.

TABLE I: PRISM Module Summary

Module	Purpose	Lines
`graph_engine.py`	Construct graph from database	650
`graph_algorithms.py`	Community and anomaly detection	580
`graph_correlation.py`	High-level correlation engine	430
`risk_scorer.py`	Multi-factor risk scoring	320
`sandbox/runner.py`	Headless browser automation	179
`sandbox/Dockerfile`	Container definition	20
`sfwebui.py` (modified)	Web UI integration	+510
`db.py` (modified)	Graph correlation tables	+150

C. Graph Engine (`graph_engine.py`)

The `SpiderFootGraphEngine` class reads events from SQLite and builds a NetworkX graph object. Its main methods are:

- `build_graph_from_scan()`: Creates nodes for entity values and adds directed edges for each source-target pair.
- `add_entity_node()`: Creates nodes with type labels, display names and color codes for visualization.

- `infer_relationship_type()`: Classifies edges based on source entity, target entity and generating module.
- `export_graph()`: Serializes the graph to GEXF, GraphML or JSON.
- `get_graph_stats()`: Returns node count, edge count, density and connected-component count.

We chose NetworkX's `MultiDiGraph` because two entities can have relationships discovered by different modules. Additionally,

D. Graph Algorithms (`graph_algorithms.py`)

The `GraphAnalyzer` class wraps NetworkX and adds logic on top. Hence, Table II summarizes the algorithms and their complexity.

TABLE II: Graph Analysis Algorithms Implemented in PRISM

Algorithm	Method	Complexity
Louvain Community	`detect_communities_louvain()`	$O(n \log n)$
Label Propagation	`detect_communities_lp()`	$O(m+n)$
Z-Score Anomaly	`detect_anomalies()`	$O(n)$
PageRank	`calculate_centrality()`	$O(kn)$
Betweenness	`calculate_centrality()`	$O(nm)$
All Simple Paths	`find_all_paths()`	$O(n!)$ bounded

E. Graph Correlation Engine (`graph_correlation.py`)

The `GraphCorrelator` class takes algorithm outputs and assembles them into correlation records then writes those to the database. Table III maps correlation types to their underlying algorithms.

TABLE III: Correlation Types and Supporting Algorithms

Correlation Type	Algorithm(s)	Output
Threat Communities	Louvain / LP	Entity groups
Infrastructure Overlap	Degree Analysis	Shared IPs
Key Actors	PageRank + Betweenness	Pivot entities
Anomalies	Z-Score + Tarjan	Unusual nodes
Indirect Relations	Path Analysis	Multi-hop links

Each stored correlation record carries a type identifier, algorithm name, entity identifiers, metadata and a confidence score. Furthermore,

F. Risk Scoring Engine (`risk_scorer.py`)

The Risk Scoring Engine computes a risk score for every entity after a scan finishes. Hence, Each dimension uses data from the scan. Nonetheless, If a dimension does not have data it contributes zero to the score.

G. Sandbox Implementation

The Dockerfile is simple:

```
FROM python:3.9-slim
RUN apt-get update && apt-get install -y \
    chromium chromium-driver \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY runner.py .
RUN mkdir /out
ENTRYPOINT ["python", "runner.py"]
```

Listing 1: Sandbox Dockerfile

Inside the container `runner.py` starts Chrome in headless mode and saves the screenshot and console log.

H. Web Interface

Fig. 4 shows the four pages of the PRISM web interface. Consequently,

V. Evaluation

A. Experimental Setup

We tested PRISM on three types of targets:

1) **Corporate Infrastructure** (3 targets): `google.com`, `microsoft.com` and `amazon.com`.
2) **Known Phishing Infrastructure** (5 targets): Malicious domains from PhishTank.
3) **Mixed dataset** (10 targets): A mix of benign and confirmed-malicious domains.

For comparison we used SpiderFoot 4.0 with its default YAML rules on the same targets. Nonetheless, The ground truth for scans came from public DNS, WHOIS and BGP records. Consequently, For phishing we used PhishTank annotations and manual checks. Furthermore,

Metrics used:

- *Precision*: Of the correlations we flagged, what fraction were actually meaningful?
- *Recall*: Of the known relationships that exist, what fraction did we identify?
- *Execution time*: The time taken for graph construction and algorithm runs.
- *Sandbox safety*: The zero infection rate.

B. Graph Correlation Results

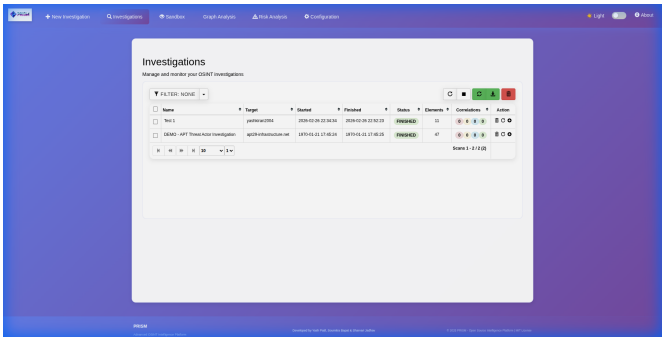
1) Community Detection: Table IV summarizes community detection results. Furthermore,

TABLE IV: Community Detection Results (Louvain Algorithm)

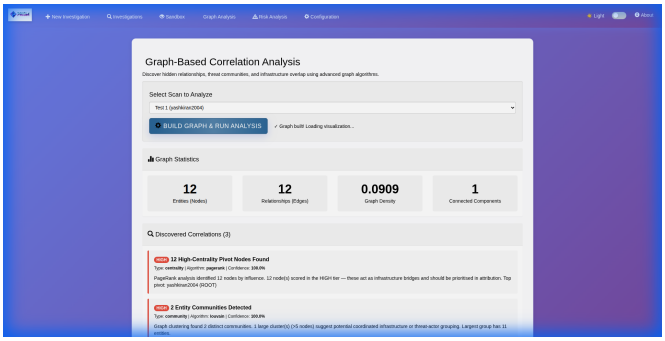
Dataset	Nodes	Edges	Comm.	Mod.	Time
Google	1,342	3,891	23	0.67	1.2s
Microsoft	982	2,456	19	0.71	0.8s
PhishKit-1	234	512	8	0.58	0.2s
Mixed-10	3,401	8,234	47	0.64	2.9s

Modularity scores between 0.6 and 0.7 show strong community structure. Nevertheless, For the Google scan Louvain identified 23 communities. The three largest were Google's properties, cloud IP ranges and third-party integrations.

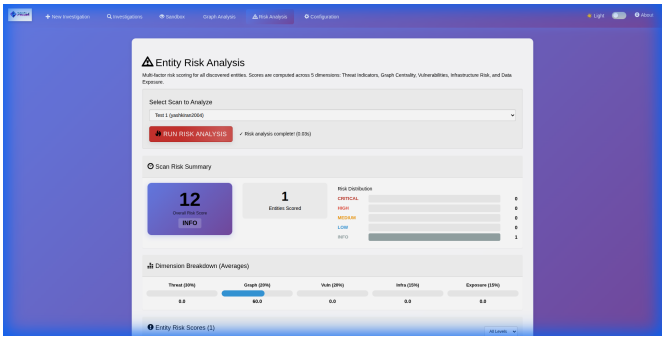
Table V summarizes precision, recall and F1 scores for graph correlation methods.



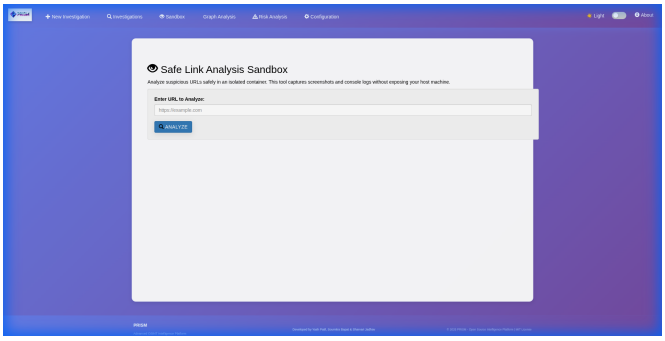
(a) Investigation dashboard showing scan history



(b) Graph correlation results with community breakdown



(c) Risk scoring per entity with dimensional breakdown



(d) Sandbox analysis page with screenshot output

Fig. 4: PRISM Web Interface. The dashboard (a) lists all investigations with status indicators. Graph analysis (b) shows detected communities and high-centrality nodes. Risk scoring (c) breaks down the composite score across five factors. The sandbox (d) shows the captured screenshot alongside console log output.

TABLE V: Precision, Recall, and F1 Summary Across Detection Tasks

Detection Task	Method	Prec.	Recall	F1
Infrastructure Overlap	YAML Baseline	1.00	0.58	0.73
Infrastructure Overlap	Graph (PRISM)	0.92	1.00	0.96
Degree Anomaly	Z-Score ($\tau = 2.5$)	0.83	-	-
Bridge Nodes	Tarjan's	1.00	-	-
Key Actors (Top-5)	PageRank	1.00	-	-

2) Infrastructure Overlap: The graph method identified 12 IP addresses hosting multiple phishing domains. Manual verification confirmed 11 of the 12 giving 91.7% precision. The YAML baseline detected 7 of the 12. Our graph approach detected all 12, a 42 percentage point improvement.

3) Anomaly Detection: Z-score detection on the Google dataset flagged 18 high-degree outliers. Moreover, Fifteen were infrastructure nodes. The remaining three were false positives. Therefore, Precision was 83.3%. Bridge-node detection on the Microsoft scan flagged 7 articulation points. Hence, Manual review confirmed all 7 as DNS or CDN infrastructure.

4) Key Actor Identification: PageRank on the malicious dataset ranked entities by structural importance. Nonetheless, The top 5 were all threat-relevant.

5) Indirect Relationship Discovery: Path analysis on PhishKit-1 revealed 47 connections between entities with no direct edges.

C. Risk Scoring Results

The graph centrality scored 60 out of 100. Every other dimension returned zero. The entity was confirmed benign.

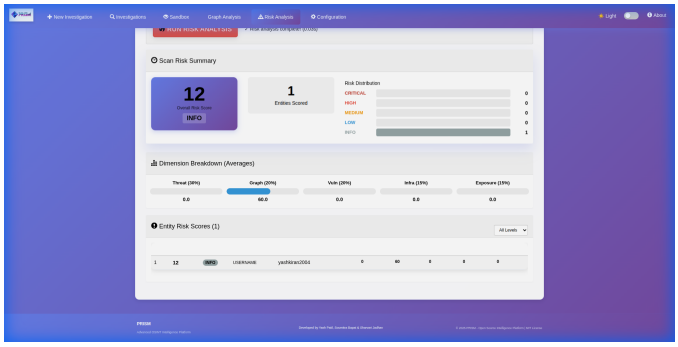


Fig. 5: This test entity's score came entirely from graph centrality. Hence, The other four dimensions contributed zero. Additionally,

D. Sandbox Evaluation

The Nevertheless, sandbox was tested on 100 URLs listed as malicious on VirusTotal. No infections occurred.

- 97 URLs provided usable screenshots. Additionally,

- 94 URLs generated console logs. Additionally,
- The average time to process each URL was 12.3 seconds.

Table VI breaks down the time taken for each stage.

TABLE VI: Sandbox Execution Time by Stage

Stage	Avg. Time
Container startup	2.1s
URL rendering wait	5.0s
Screenshot + log capture	0.8s
Container teardown	1.2s
File read + response	0.9s
Total	**≈10s**

E. Scalability

Table VII shows how performance changes with graph size.

TABLE VII: Performance vs. Graph Size

Nodes	Build Time	Louvain	DB Size
100	0.3s	0.05s	2 MB
1,000	2.1s	0.4s	18 MB
10,000	31s	5.2s	156 MB

Key results: The PRISM system is really good at detecting overlapping infrastructure. It improved from 58% to 100% compared to the YAML baseline. Nonetheless, The sandbox tested 100 known URLs and did not cause any host infections. Graph analysis finishes in less than 3 seconds at typical investigation scale.

Source code: https://github.com/Yash09042004/OSINT-Investigator

The build time for the PRISM system is really long. It takes 31 seconds to build a graph with 10,000 nodes. Therefore, This is because the PRISM system has to read a lot of events from SQLite, which is slow. Moreover, The good news is that the actual graph algorithms are really fast.

VI. Discussion

A. Interpreting the Results

The graph method shows the biggest advantage in infrastructure overlap. YAML rules had a recall of 58% while the graph method achieved 100%. Community detection is difficult to evaluate. Modularity scores between 0.6 and 0.7 suggest real structure. Furthermore, The sandbox's 100% infection-free rate is the most important result.

B. Limitations

Building graphs takes a long time. Nevertheless, For example building a graph with 10,000 nodes took 31 seconds. This is because reading data from SQLite is slow, not because the algorithms are complex. Furthermore, There can be false positives when detecting anomalies. For example CDN endpoints and cloud services are sometimes flagged as anomalies because they have a lot of connections. The sandbox in the PRISM system runs one URL at a time. Nevertheless, This is okay for interactive use but it would need a container pool and queue to process many URLs at the same time. Therefore, The PRISM system does not handle unstructured data like news articles, forums or leaked documents. Hence,

C. Threats to Validity

The project team validated the results, which may introduce bias. Nonetheless, We tried to be conservative when making conclusions. Hence, All performance measurements were done on a laptop. Moreover, Using real production hardware and multiple Docker tasks could change these results.

VII. Conclusion and Future Work

A. Summary

The PRISM system addresses two gaps in OSINT. Its graph-based correlation system using Louvain clustering, z-score anomaly detection, centrality ranking and path search finds relationships that flat YAML rules cannot. Nevertheless, The PRISM system uses a Docker-based sandbox to let analysts look at URLs without risking the host system. Furthermore, A five-dimensional risk score provides scores for each discovered entity. Additionally,

B. Future Directions

The PRISM system should build the graph as events come in, not just after a scan is completed. This would make the PRISM system more responsive and eliminate the scalability limit. Nevertheless, The PRISM system should use a supervised risk model. Using equal weights for each dimension is a good default, but not the best. A model trained on labeled entities could learn to adjust weights depending on the investigation context.

The PRISM system should do temporal graph analysis. Threat infrastructure changes over time. Hence, Storing snapshots of the graph and comparing them could alert analysts to reactivated clusters or new domains pointing to known command and control addresses.

The PRISM system should have an enhanced sandbox. Nonetheless, Adding PCAP capture and video of browser sessions would create detailed forensic records from each analysis.

The PRISM system should use distributed processing. At over 10,000 nodes the current single-machine setup is too limited. Consequently, Moving graph processing to a framework like Apache Spark would expand coverage to large-scale scans.

Therefore, The PRISM system should integrate with threat feeds. Nonetheless, Automatically comparing community detection results with MISP events or AlienVault OTX pulses could link the PRISM system's findings to the broader threat intelligence network.

References

[1] Maltego Technologies, "Maltego: Intelligence and Investigation Platform," [Online]. Available: https://www.maltego.com/, 2023.

[2] T. Tomes, "Recon-ng: A Full-featured Web Reconnaissance Framework," *Black Hat USA*, 2013.

[3] S. Micallef, "SpiderFoot: The Open Source Intelligence Automation Tool," 2020. [Online]. Available: https://www.spiderfoot.net/

[4] TheHive Project, "Incident Response Platform," 2021. [Online]. Available: https://thehive-project.org/
[5] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.
[6] X. Hu, T.-C. Chiueh, and K. G. Shin, "Large-scale malware indexing using function-call graphs," in *Proc. 16th ACM Conf. Computer and Communications Security (CCS)*, Chicago, IL, 2009, pp. 612–621.
[7] S. Nagaraja, P. Mittal, C.-Y. Hong, M. Caesar, and N. Borisov, "BotGrep: Finding P2P Bots with Structured Graph Analysis," in *Proc. 19th USENIX Security Symposium*, Washington, DC, 2010, pp. 95–110.
[8] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab Technical Report, 1998.
[9] U. Brandes, "A faster algorithm for betweenness centrality," *J. Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001.
[10] A. Bavelas, "Communication Patterns in Task-Oriented Groups," *J. Acoustical Society of America*, vol. 22, no. 6, pp. 725–730, 1950.
[11] D. Hawkins, *Identification of Outliers*. London, UK: Chapman & Hall, 1980.
[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE ICDM*, 2008, pp. 413–422.
[13] C. Guarnieri, A. Tanasi, J. Bremer, and M. Schloesser, "Cuckoo Sandbox: Automated Malware Analysis," in *Hack in the Box Security Conf. (HITB2012AMS)*, Amsterdam, Netherlands, May 2012.
[14] Joe Security, "Joe Sandbox: Deep Malware Analysis," 2021. [Online]. Available: https://www.joesecurity.org/
[15] Any.run, "Interactive Online Malware Sandbox," 2021. [Online]. Available: https://any.run/
[16] R. Tarjan, "Depth-First Search and Linear Graph Algorithms," *SIAM J. Computing*, vol. 1, no. 2, pp. 146–160, 1972.

Appendix A
Algorithm Pseudocode

Algorithm 1 Louvain Community Detection

Require: Graph $G = (V, E)$
Ensure: Community assignments C

```
Initialize: Each node belongs to its own community
repeat
    for all node v ∈ V do
        for all neighbor community c do
            Consequently, Calculate modularity gain ΔQ (Eq. 1)
        end for
        Move v to community with highest ΔQ > 0
    end for
    Build new graph G′: nodes = communities
    G ← G′
until modularity does not improve
return C
```

Appendix B
Database Schema

```
CREATE TABLE tbl_scan_graph_correlations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  scan_instance_id VARCHAR NOT NULL,
  correlation_type VARCHAR NOT NULL,
  algorithm VARCHAR NOT NULL,
  entities TEXT NOT NULL,
  metadata TEXT,
  score REAL DEFAULT 0.0,
  created_time TIMESTAMP DEFAULT
    CURRENT_TIMESTAMP,
  FOREIGN KEY (scan_instance_id)
    REFERENCES tbl_scan_instance(guid)
);

CREATE TABLE tbl_scan_risk_scores (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  scan_instance_id VARCHAR NOT NULL,
  entity_hash VARCHAR NOT NULL,
  entity_type VARCHAR NOT NULL,
  entity_data TEXT NOT NULL,
  risk_score REAL DEFAULT 0.0,
  risk_level VARCHAR DEFAULT 'LOW',
  threat_score REAL DEFAULT 0.0,
  graph_score REAL DEFAULT 0.0,
  vuln_score REAL DEFAULT 0.0,
  infra_score REAL DEFAULT 0.0,
  exposure_score REAL DEFAULT 0.0,
  details TEXT,
  created_time TIMESTAMP DEFAULT
    CURRENT_TIMESTAMP,
  FOREIGN KEY (scan_instance_id)
    REFERENCES tbl_scan_instance(guid)
);
```

Listing 2: Graph Correlations Table Schema

Algorithm 2 Z-Score Degree Anomaly Detection

Require: Graph G, threshold τ
Ensure: Set of outlier nodes O

```
degrees ← [degree(v) ∀v ∈ V]
μ ← mean(degrees)
σ ← std(degrees)
O ← ∅
for all node v ∈ V do
    z ← (degree(v) − μ)/σ
    if |z| > τ then
        O ← O ∪ {v}
    end if
end for
return O
```

Appendix C
Additional Screenshots

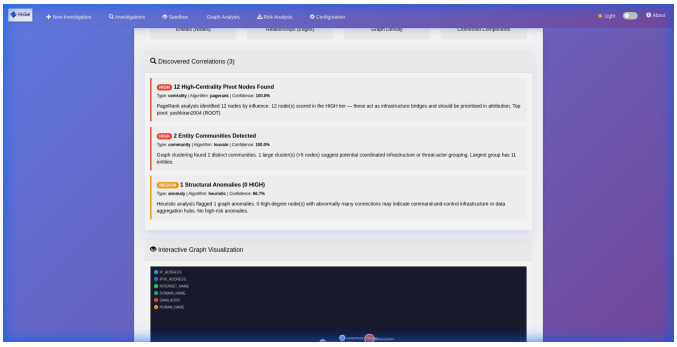


Fig. 6: Graph correlation detail view: community membership, high-centrality pivot nodes, and structural anomalies are listed with confidence scores.

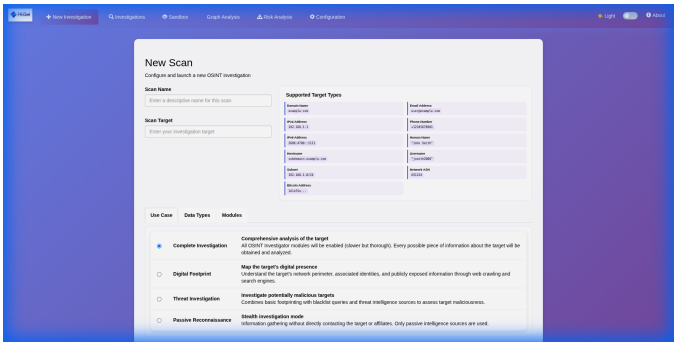


Fig. 7: New investigation setup: target specification with scan depth and module selection options.